Java 2D Array Quiz – Ausführliche Analyse der 23 Fragen

Benjamin Malicke ©

26. Oktober 2025

Inhaltsverzeichnis

1 Einleitung **1**

2 Frage 0 **1**
2.1 Code . . . 1
2.2 Initialisierung . . . 1
2.3 Schritt-für-Schritt-Ausführung . . . 1
2.4 Ausgabe . . . 2
2.5 Zusätzliche Erklärungen . . . 2

3 Frage 1 **2**
3.1 Code . . . 2
3.2 Initialisierung . . . 2
3.3 Schritt-für-Schritt-Ausführung . . . 2
3.4 Ausgabe . . . 2
3.5 Zusätzliche Erklärungen . . . 2

4 Frage 2 **2**
4.1 Code . . . 2
4.2 Initialisierung . . . 3
4.3 Schritt-für-Schritt-Ausführung . . . 3
4.4 Ausgabe . . . 3
4.5 Zusätzliche Erklärungen . . . 3

5 Frage 3 **3**
5.1 Code . . . 3
5.2 Initialisierung . . . 3
5.3 Schritt-für-Schritt-Ausführung . . . 3
5.4 Ausgabe . . . 3
5.5 Zusätzliche Erklärungen . . . 3

6 Frage 4 **4**
6.1 Code . . . 4
6.2 Initialisierung . . . 4
6.3 Schritt-für-Schritt-Ausführung . . . 4
6.4 Ausgabe . . . 4
6.5 Zusätzliche Erklärungen . . . 4

7 Frage 5 **4**
7.1 Code . . . 4
7.2 Initialisierung . . . 4
7.3 Schritt-für-Schritt-Ausführung . . . 4
7.4 Ausgabe . . . 5
7.5 Zusätzliche Erklärungen . . . 5

8 Frage 6 **5**
8.1 Code . . . 5
8.2 Initialisierung . . . 5
8.3 Schritt-für-Schritt-Ausführung . . . 5
8.4 Ausgabe . . . 5
8.5 Zusätzliche Erklärungen . . . 5

9 Frage 7 **5**
9.1 Code . . . 5
9.2 Initialisierung . . . 5
9.3 Schritt-für-Schritt-Ausführung . . . 6
9.4 Ausgabe . . . 6
9.5 Zusätzliche Erklärungen . . . 6

10 Frage 8 **6**
10.1 Code . . . 6
10.2 Initialisierung . . . 6
10.3 Schritt-für-Schritt-Ausführung . . . 6
10.4 Ausgabe . . . 6
10.5 Zusätzliche Erklärungen . . . 6

11 Frage 9 **7**
11.1 Code . . . 7
11.2 Initialisierung . . . 7
11.3 Schritt-für-Schritt-Ausführung . . . 7
11.4 Ausgabe . . . 7
11.5 Zusätzliche Erklärungen . . . 7

12 Frage 10 **7**
12.1 Code . . . 7
12.2 Initialisierung . . . 7
12.3 Schritt-für-Schritt-Ausführung . . . 7
12.4 Ausgabe . . . 7
12.5 Zusätzliche Erklärungen . . . 8

13 Frage 11 **8**
13.1 Code . . . 8
13.2 Initialisierung . . . 8
13.3 Schritt-für-Schritt-Ausführung . . . 8
13.4 Ausgabe . . . 8
13.5 Zusätzliche Erklärungen . . . 8

14 Frage 12 **8**
14.1 Code . . . 8
14.2 Initialisierung . . . 9
14.3 Schritt-für-Schritt-Ausführung . . . 9
14.4 Ausgabe . . . 9
14.5 Zusätzliche Erklärungen . . . 9

15 Frage 13 **9**
15.1 Code . . . 9
15.2 Initialisierung . . . 9
15.3 Schritt-für-Schritt-Ausführung . . . 9
15.4 Ausgabe . . . 9
15.5 Zusätzliche Erklärungen . . . 9

16 Frage 14 **9**
16.1 Code . . . 9
16.2 Initialisierung . . . 10
16.3 Schritt-für-Schritt-Ausführung . . . 10
16.4 Ausgabe . . . 10
16.5 Zusätzliche Erklärungen . . . 10

17 Frage 15 **10**
17.1 Code . . . 10
17.2 Initialisierung . . . 10
17.3 Schritt-für-Schritt-Ausführung . . . 10
17.4 Ausgabe . . . 10
17.5 Zusätzliche Erklärungen . . . 10

18 Frage 16 **11**
18.1 Code . . . 11
18.2 Initialisierung . . . 11
18.3 Schritt-für-Schritt-Ausführung . . . 11
18.4 Ausgabe . . . 11
18.5 Zusätzliche Erklärungen . . . 11

19 Frage 17 **11**
19.1 Code . . . 11
19.2 Initialisierung . . . 11
19.3 Schritt-für-Schritt-Ausführung . . . 11
19.4 Ausgabe . . . 12
19.5 Zusätzliche Erklärungen . . . 12

20 Frage 18 **12**
20.1 Code . . . 12
20.2 Initialisierung . . . 12
20.3 Schritt-für-Schritt-Ausführung . . . 12
20.4 Ausgabe . . . 12
20.5 Zusätzliche Erklärungen . . . 12

21 Frage 19 **12**
21.1 Code . . . 12
21.2 Initialisierung . . . 12
21.3 Schritt-für-Schritt-Ausführung . . . 13
21.4 Ausgabe . . . 13
21.5 Zusätzliche Erklärungen . . . 13

22 Frage 20 **13**
22.1 Code . . . 13
22.2 Initialisierung . . . 13
22.3 Schritt-für-Schritt-Ausführung . . . 13
22.4 Ausgabe . . . 13
22.5 Zusätzliche Erklärungen . . . 14

23 Frage 21 **14**
23.1 Code . . . 14
23.2 Initialisierung . . . 14
23.3 Schritt-für-Schritt-Ausführung . . . 14
23.4 Ausgabe . . . 14
23.5 Zusätzliche Erklärungen . . . 14

24 Frage 22 **14**
24.1 Code . . . 14
24.2 Initialisierung . . . 14
24.3 Schritt-für-Schritt-Ausführung . . . 15
24.4 Ausgabe . . . 15
24.5 Zusätzliche Erklärungen . . . 15

1 Einleitung

Dieses Dokument analysiert die 23 Fragen des Java 2D Array Quiz. Jede Frage besteht aus einem Java-Code-Snippet, das mit zweidimensionalen Arrays arbeitet. Wir präsentieren den Code, führen eine schrittweise Ausführung (Dry-Run) durch und erklären die Ergebnisse ausführlich auf Deutsch. Ziel ist es, ein tiefes Verständnis der Konzepte rund um 2D-Arrays zu fördern, einschließlich häufiger Fehler wie ArrayIndexOutOfBoundsException oder unsicherer Schleifengrenzen.

Die Fragen sind von 0 bis 22 nummeriert. Für jede Frage:

- **Code:** Der vollständige Java-Code.
- **Initialisierung:** Beschreibung des anfänglichen Zustands.
- **Schritt-für-Schritt-Ausführung:** Detaillierte Analyse jeder Operation.
- **Ausgabe:** Das Endergebnis und warum es so ist.
- **Zusätzliche Erklärungen:** Hinweise zu 2D-Arrays, Schleifen, Fehlerquellen und Best Practices.

2 Frage 0

2.1 Code

```java
int[][] arr = {{1,2,3},{4,5,6}};
int sum = 0;
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    sum += arr[i][j];
  }
}
System.out.print(sum);
```

2.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, sum=0. arr.length=2 (Zeilen), arr[0].length=3 (Spalten pro Zeile).

2.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += arr[0][0] = 0 + 1 = 1.
2. i=0, j=1: sum += arr[0][1] = 1 + 2 = 3.
3. i=0, j=2: sum += arr[0][2] = 3 + 3 = 6.
4. i=1, j=0: sum += arr[1][0] = 6 + 4 = 10.
5. i=1, j=1: sum += arr[1][1] = 10 + 5 = 15.
6. i=1, j=2: sum += arr[1][2] = 15 + 6 = 21.
7. `System.out.print(sum);` gibt 21 aus.

2.4 Ausgabe

Ausgabe: 21

2.5 Zusätzliche Erklärungen

Verschachtelte Schleifen durchlaufen alle Elemente eines 2x3-Arrays. `arr.length` gibt die Anzahl der Zeilen, `arr[0].length` die Anzahl der Spalten in der ersten Zeile. Die Summe aller Elemente ist 1+2+3+4+5+6=21.

3 Frage 1

3.1 Code

```
int[][] arr = {{1,2},{3,4}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr.length; j++) {
    System.out.print(arr[i][j]);
  }
}
```

3.2 Initialisierung

2D-Array `arr` = {{1,2},{3,4}}, arr.length=2.

3.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[0][1] = 2.
3. i=1, j=0: print arr[1][0] = 3.
4. i=1, j=1: print arr[1][1] = 4.

3.4 Ausgabe

Ausgabe: 1234

3.5 Zusätzliche Erklärungen

Die innere Schleife verwendet `arr.length` (2) statt `arr[0].length` (2). Da das Array quadratisch ist (2x2), funktioniert dies hier, ist aber unsicher, da `arr[i].length` variieren kann. Besser wäre `arr[i].length` für die innere Schleife.

4 Frage 2

4.1 Code

```
int[][] arr = {{2,4},{6,8}};
int i=0, j=0, sum=0;
while(i<arr.length) {
  j=0;
  while(j<arr[0].length) {
    sum += arr[i][j];
    j++;
  }
  i++;
}
System.out.print(sum);
```

4.2 Initialisierung

2D-Array `arr` = {{2,4},{6,8}}, sum=0, i=0, j=0.

4.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += arr[0][0] = 0 + 2 = 2, j++=1.
2. i=0, j=1: sum += arr[0][1] = 2 + 4 = 6, j++=2.
3. i=0, j=2: j<arr[0].length (2) → Ende innere Schleife.

4. i++=1.
5. i=1, j=0: sum += arr[1][0] = 6 + 6 = 12, j++=1.
6. i=1, j=1: sum += arr[1][1] = 12 + 8 = 20, j++=2.
7. i=1, j=2: j<arr[0].length (2) → Ende innere Schleife.
8. i++=2, i<arr.length (2) → Ende äußere Schleife.
9. `System.out.print(sum);` gibt 20 aus.

4.4 Ausgabe

Ausgabe: 20

4.5 Zusätzliche Erklärungen

While-Schleifen statt for-Schleifen, aber gleiche Logik wie Frage 0. Summiert alle Elemente: 2+4+6+8=20.

5 Frage 3

5.1 Code

```
int[][] arr = {{1,2},{3,4}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    System.out.print(arr[j][i]);
  }
}
```

5.2 Initialisierung

2D-Array `arr` = {{1,2},{3,4}}.

5.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[1][0] = 3.
3. i=1, j=0: print arr[0][1] = 2.
4. i=1, j=1: print arr[1][1] = 4.

5.4 Ausgabe

Ausgabe: 1324

5.5 Zusätzliche Erklärungen

Die Indizes sind vertauscht (`arr[j][i]` statt `arr[i][j]`), was eine transponierte Ausgabe ergibt. Das Array wird spaltenweise statt zeilenweise ausgegeben.

6 Frage 4

6.1 Code

```java
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    if(arr[i][j] % 2 == 0) {
      System.out.print(arr[i][j]);
    }
  }
}
```

6.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}.

6.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: arr[0][0]=1, 1%2=1 → skip.
2. i=0, j=1: arr[0][1]=2, 2%2=0 → print 2.
3. i=0, j=2: arr[0][2]=3, 3%2=1 → skip.
4. i=1, j=0: arr[1][0]=4, 4%2=0 → print 4.
5. i=1, j=1: arr[1][1]=5, 5%2=1 → skip.
6. i=1, j=2: arr[1][2]=6, 6%2=0 → print 6.

6.4 Ausgabe

Ausgabe: 246

6.5 Zusätzliche Erklärungen

Gibt nur gerade Elemente aus (2,4,6). Modulo prüft Parität. Schleifen durchlaufen das Array zeilenweise.

7 Frage 5

7.1 Code

```java
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr.length; j++) {
    System.out.print(arr[i][j]);
  }
}
```

7.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, arr.length=2.

7.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[0][1] = 2.
3. i=1, j=0: print arr[1][0] = 4.
4. i=1, j=1: print arr[1][1] = 5.

7.4 Ausgabe

Ausgabe: 1245

7.5 Zusätzliche Erklärungen

Die innere Schleife verwendet `arr.length` (2) statt `arr[0].length` (3), wodurch die dritte Spalte (arr[i][2]) ignoriert wird. Dies ist unsicher für nicht-quadratische Arrays, da es unvollständige Ergebnisse liefert.

8 Frage 6

8.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    if(i == j) {
      System.out.print(arr[i][j]);
    }
  }
}
```

8.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}.

8.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: i==j → print arr[0][0] = 1.
2. i=0, j=1: i!=j → skip.
3. i=0, j=2: i!=j → skip.
4. i=1, j=0: i!=j → skip.
5. i=1, j=1: i==j → print arr[1][1] = 5.
6. i=1, j=2: i!=j → skip.

8.4 Ausgabe

Ausgabe: 15

8.5 Zusätzliche Erklärungen

Gibt nur Elemente auf der Hauptdiagonale (i==j) aus: arr[0][0]=1, arr[1][1]=5.

9 Frage 7

9.1 Code

```
int[][] arr = {{2,4},{6,8}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    System.out.print(arr[i][j] * arr[j][i]);
  }
}
```

9.2 Initialisierung

2D-Array `arr` = {{2,4},{6,8}}.

9.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: arr[0][0]*arr[0][0] = 2*2 = 4 → print 4.
2. i=0, j=1: arr[0][1]*arr[1][0] = 4*6 = 24 → print 24.
3. i=1, j=0: arr[1][0]*arr[0][1] = 6*4 = 24 → print 24.
4. i=1, j=1: arr[1][1]*arr[1][1] = 8*8 = 64 → print 64.

9.4 Ausgabe

Ausgabe: 4242464

9.5 Zusätzliche Erklärungen

Multipliziert Elemente mit ihren transponierten Gegenstücken (`arr[i][j] * arr[j][i]`). Ergibt eine symmetrische Ausgabe für i,j und j,i.

10 Frage 8

10.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[1].length; j++) {
    System.out.print(arr[i][j]);
  }
}
```

10.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, arr[1].length=3.

10.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[0][1] = 2.
3. i=0, j=2: print arr[0][2] = 3.
4. i=1, j=0: print arr[1][0] = 4.
5. i=1, j=1: print arr[1][1] = 5.
6. i=1, j=2: print arr[1][2] = 6.

10.4 Ausgabe

Ausgabe: 123456

10.5 Zusätzliche Erklärungen

Gibt alle Elemente zeilenweise aus. `arr[1].length` ist korrekt, aber `arr[i].length` ist sicherer, um unterschiedliche Zeilenlängen zu handhaben.

11 Frage 9

11.1 Code

```
int[][] arr = {{1,2},{3,4}};
int sum = 0;
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    sum += arr[i][j];
  }
}
System.out.print(sum);
```

11.2 Initialisierung

2D-Array `arr` = {{1,2},{3,4}}, sum=0.

11.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += arr[0][0] = 0 + 1 = 1.
2. i=0, j=1: sum += arr[0][1] = 1 + 2 = 3.
3. i=1, j=0: sum += arr[1][0] = 3 + 3 = 6.
4. i=1, j=1: sum += arr[1][1] = 6 + 4 = 10.

11.4 Ausgabe

Ausgabe: 10

11.5 Zusätzliche Erklärungen

Summiert alle Elemente eines 2x2-Arrays: 1+2+3+4=10.

12 Frage 10

12.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr.length; j++) {
    System.out.print(arr[j][i]);
  }
}
```

12.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, arr.length=2.

12.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[1][0] = 4.
3. i=1, j=0: print arr[0][1] = 2.
4. i=1, j=1: print arr[1][1] = 5.

12.4 Ausgabe

Ausgabe: 1425

12.5 Zusätzliche Erklärungen

Transponierte Ausgabe durch vertauschte Indizes (`arr[j][i]`). Die innere Schleife verwendet `arr.length` (2) statt `arr[0].length` (3), wodurch die dritte Spalte ignoriert wird.

13 Frage 11

13.1 Code

```
int[][] arr = {{1,2},{3,4}};
int i=0, sum=0;
while(i<arr.length) {
  int j=0;
  while(j<arr[0].length) {
    sum += arr[i][j];
    j++;
  }
  i++;
}
System.out.print(sum);
```

13.2 Initialisierung

2D-Array `arr` = {{1,2},{3,4}}, sum=0, i=0.

13.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += arr[0][0] = 0 + 1 = 1, j++=1.
2. i=0, j=1: sum += arr[0][1] = 1 + 2 = 3, j++=2.
3. i=0, j=2: j<arr[0].length (2) → Ende innere Schleife.
4. i++=1.
5. i=1, j=0: sum += arr[1][0] = 3 + 3 = 6, j++=1.
6. i=1, j=1: sum += arr[1][1] = 6 + 4 = 10, j++=2.
7. i=1, j=2: j<arr[0].length (2) → Ende innere Schleife.
8. i++=2, i<arr.length (2) → Ende äußere Schleife.

13.4 Ausgabe

Ausgabe: 10

13.5 Zusätzliche Erklärungen

Äquivalent zu Frage 9, aber mit while-Schleifen. Summiert alle Elemente: 1+2+3+4=10.

14 Frage 12

14.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    arr[i][j] = i + j;
  }
}
System.out.print(arr[1][2]);
```

14.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}.

14.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: arr[0][0] = 0 + 0 = 0.
2. i=0, j=1: arr[0][1] = 0 + 1 = 1.
3. i=0, j=2: arr[0][2] = 0 + 2 = 2.
4. i=1, j=0: arr[1][0] = 1 + 0 = 1.
5. i=1, j=1: arr[1][1] = 1 + 1 = 2.
6. i=1, j=2: arr[1][2] = 1 + 2 = 3.
7. `System.out.print(arr[1][2]);` gibt 3 aus.

14.4 Ausgabe

Ausgabe: 3

14.5 Zusätzliche Erklärungen

Überschreibt jedes Element mit der Summe seiner Indizes (i+j). Nur arr[1][2] wird ausgegeben.

15 Frage 13

15.1 Code

```
int[][] arr = {{1,2},{3,4},{5,6}};
System.out.print(arr[0].length);
```

15.2 Initialisierung

2D-Array `arr` = {{1,2},{3,4},{5,6}}.

15.3 Schritt-für-Schritt-Ausführung

1. arr[0].length = 2 (Anzahl der Spalten in der ersten Zeile).
2. `System.out.print(2);` gibt 2 aus.

15.4 Ausgabe

Ausgabe: 2

15.5 Zusätzliche Erklärungen

`arr[0].length` gibt die Länge der ersten Zeile. `arr.length` (3) gibt die Anzahl der Zeilen.

16 Frage 14

16.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    System.out.print(arr[i][2-j]);
  }
}
```

16.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}.

16.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][2-0] = arr[0][2] = 3.
2. i=0, j=1: print arr[0][2-1] = arr[0][1] = 2.
3. i=0, j=2: print arr[0][2-2] = arr[0][0] = 1.
4. i=1, j=0: print arr[1][2-0] = arr[1][2] = 6.
5. i=1, j=1: print arr[1][2-1] = arr[1][1] = 5.
6. i=1, j=2: print arr[1][2-2] = arr[1][0] = 4.

16.4 Ausgabe

Ausgabe: 321654

16.5 Zusätzliche Erklärungen

Die Spaltenindizes werden umgekehrt (2-j), was jede Zeile rückwärts ausgibt.

17 Frage 15

17.1 Code

```
int[][] arr = {{1,2},{3,4}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    if(arr[i][j] % 2 == 0) {
      System.out.print(arr[i][j]);
    }
  }
}
```

17.2 Initialisierung

2D-Array `arr` = {{1,2},{3,4}}.

17.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: arr[0][0]=1, 1%2=1 → skip.
2. i=0, j=1: arr[0][1]=2, 2%2=0 → print 2.
3. i=1, j=0: arr[1][0]=3, 3%2=1 → skip.
4. i=1, j=1: arr[1][1]=4, 4%2=0 → print 4.

17.4 Ausgabe

Ausgabe: 24

17.5 Zusätzliche Erklärungen

Gibt nur gerade Elemente (2,4) zeilenweise aus.

18 Frage 16

18.1 Code

```
int[][] arr = {{1,3,5},{2,4,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr.length; j++) {
    System.out.print(arr[i][j]);
  }
}
```

18.2 Initialisierung

2D-Array `arr` = {{1,3,5},{2,4,6}}, arr.length=2.

18.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[0][1] = 3.
3. i=1, j=0: print arr[1][0] = 2.
4. i=1, j=1: print arr[1][1] = 4.

18.4 Ausgabe

Ausgabe: 1324

18.5 Zusätzliche Erklärungen

Die innere Schleife verwendet `arr.length` (2) statt `arr[0].length` (3), wodurch die dritte Spalte ignoriert wird. Unsicher für nicht-quadratische Arrays.

19 Frage 17

19.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
int sum = 0;
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    sum += arr[i][j];
  }
}
System.out.print(sum);
```

19.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, sum=0.

19.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += arr[0][0] = 0 + 1 = 1.
2. i=0, j=1: sum += arr[0][1] = 1 + 2 = 3.
3. i=0, j=2: sum += arr[0][2] = 3 + 3 = 6.
4. i=1, j=0: sum += arr[1][0] = 6 + 4 = 10.
5. i=1, j=1: sum += arr[1][1] = 10 + 5 = 15.
6. i=1, j=2: sum += arr[1][2] = 15 + 6 = 21.

19.4 Ausgabe

Ausgabe: 21

19.5 Zusätzliche Erklärungen

Identisch zu Frage 0. Summiert alle Elemente: 1+2+3+4+5+6=21.

20 Frage 18

20.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr.leght; j++) {
    System.out.print(arr[i][j]);
  }
}
```

20.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}.

20.3 Schritt-für-Schritt-Ausführung

1. Der Code enthält einen Tippfehler: `arr.leght` statt `arr.length`.
2. Dies führt zu einem Compiler-Fehler (Syntaxfehler).

20.4 Ausgabe

Ausgabe: Compiler-Fehler

20.5 Zusätzliche Erklärungen

Tippfehler verhindern die Ausführung. Korrekt wäre `arr[0].length` oder `arr[i].length` für die innere Schleife.

21 Frage 19

21.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
int i=0, sum=1;
while(i<arr.length) {
  int j=0;
  while(j<arr[0].length) {
    sum *= arr[i][j];
    j++;
  }
  i++;
}
System.out.print(sum);
```

21.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, sum=1, i=0.

21.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum *= arr[0][0] = 1 * 1 = 1, j++=1.
2. i=0, j=1: sum *= arr[0][1] = 1 * 2 = 2, j++=2.
3. i=0, j=2: sum *= arr[0][2] = 2 * 3 = 6, j++=3.
4. i=0, j=3: j<arr[0].length (3) → Ende innere Schleife.
5. i++=1.
6. i=1, j=0: sum *= arr[1][0] = 6 * 4 = 24, j++=1.
7. i=1, j=1: sum *= arr[1][1] = 24 * 5 = 120, j++=2.
8. i=1, j=2: sum *= arr[1][2] = 120 * 6 = 720, j++=3.
9. i=1, j=3: j<arr[0].length (3) → Ende innere Schleife.
10. i++=2, i<arr.length (2) → Ende äußere Schleife.

21.4 Ausgabe

Ausgabe: 720

21.5 Zusätzliche Erklärungen

Berechnet das Produkt aller Elemente: 1*2*3*4*5*6=720. Initialisierung mit sum=1 ist wichtig, um eine korrekte Multiplikation zu gewährleisten.

22 Frage 20

22.1 Code

```
int[][] arr = {{1,2,3},{4,5,6}};
for(int i=0; i<arr[0].length; i++) {
  for(int j=0; j<arr.length; j++) {
    System.out.print(arr[j][i]);
  }
}
```

22.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6}}, arr[0].length=3, arr.length=2.

22.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: print arr[0][0] = 1.
2. i=0, j=1: print arr[1][0] = 4.
3. i=1, j=0: print arr[0][1] = 2.
4. i=1, j=1: print arr[1][1] = 5.
5. i=2, j=0: print arr[0][2] = 3.
6. i=2, j=1: print arr[1][2] = 6.

22.4 Ausgabe

Ausgabe: 142536

22.5 Zusätzliche Erklärungen

Die Schleifen sind umgedreht: Äußere Schleife über Spalten (i), innere über Zeilen (j). Ergibt eine spaltenweise (transponierte) Ausgabe.

23 Frage 21

23.1 Code

```
int[][] arr = {{1,0,3},{4,5,6}};
int sum = 0;
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr[0].length; j++) {
    sum += 10 / arr[i][j];
  }
}
System.out.print(sum);
```

23.2 Initialisierung

2D-Array `arr` = {{1,0,3},{4,5,6}}, sum=0.

23.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += 10/arr[0][0] = 10/1 = 10 → sum = 10.
2. i=0, j=1: sum += 10/arr[0][1] = 10/0 → ArithmeticException (Division durch Null).
3. Programm stoppt.

23.4 Ausgabe

Ausgabe: ArithmeticException

23.5 Zusätzliche Erklärungen

Division durch Null (arr[0][1]=0) verursacht eine Laufzeitexception. Vor der Division sollte geprüft werden, ob arr[i][j] $\neq$ 0 ist.

24 Frage 22

24.1 Code

```
int[][] arr = {{1,2,3},{4,5,6},{6}};
int sum = 0;
for(int i=0; i<arr.length; i++) {
  for(int j=0; j<arr.length; j++) {
    sum += arr[i][j];
  }
}
System.out.print(sum);
```

24.2 Initialisierung

2D-Array `arr` = {{1,2,3},{4,5,6},{6}}, sum=0, arr.length=3.

24.3 Schritt-für-Schritt-Ausführung

1. i=0, j=0: sum += arr[0][0] = 0 + 1 = 1.
2. i=0, j=1: sum += arr[0][1] = 1 + 2 = 3.
3. i=0, j=2: sum += arr[0][2] = 3 + 3 = 6.
4. i=1, j=0: sum += arr[1][0] = 6 + 4 = 10.
5. i=1, j=1: sum += arr[1][1] = 10 + 5 = 15.
6. i=1, j=2: sum += arr[1][2] = 15 + 6 = 21.
7. i=2, j=0: sum += arr[2][0] = 21 + 6 = 27.
8. i=2, j=1: arr[2][1] → ArrayIndexOutOfBoundsException (arr[2] hat nur ein Element).
9. Programm stoppt.

24.4 Ausgabe

Ausgabe: ArrayIndexOutOfBoundsException

24.5 Zusätzliche Erklärungen

Die innere Schleife verwendet `arr.length` (3) statt `arr[i].length`. Da arr[2].length=1 ist, führt j=1 zu einer Exception. Korrekt wäre `arr[i].length`.